

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №3
по дисциплине «Архитектура суперкомпьютеров»
на тему «Модель кэш-памяти прямого отображения»
Вариант 7

Выполнила студентка группы
№5130201/30001
Проверил

Лаишевкина А.М. _____
Чуватов М.В. _____

«____» _____ 2025г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Первая часть работы	4
1.1 Постановка задачи	4
1.2 Ход выполнения работы	5
1.3 Заключение первой части работы	7
2 Вторая часть работы	9
2.1 Постановка задачи	9
2.2 Ход выполнения работы	9
2.3 Заключение второй части работы	10
Заключение	11
Список использованных источников	12
Приложение А. Результат первой части работы	13
Приложение Б. Результат второй части работы	14

Введение

В отчете представлено описание хода выполнения лабораторной работы по дисциплине «Архитектура суперкомпьютеров».

Работа представляет собой разработку модели кэш-памяти прямого отображения. Кэш-память прямого отображения - это способ организации кэш-памяти в процессорах, при котором каждая строка основной памяти может быть помещена только в одно конкретное место кэша. Каждая строка кэша содержит тег (для идентификации данных), сами данные и флаг валидности. При обращении процессора к памяти сначала определяется соответствующая строка кэша, после чего проверяется совпадение тега: в случае успеха данные считываются из кэша (попадание), в противном случае происходит загрузка из RAM (промах). Основными преимуществами данной организации являются высокая скорость работы и минимальные накладные расходы, однако возможны частые конфликты при обращении к разным адресам, отображаемым в одну и ту же строку кэша.

1 Первая часть работы

1.1 Постановка задачи

В первой части данной лабораторной работы необходимо составить модель кэш-памяти прямого отображения в среде logisim-evolution со следующими свойствами:

1. Побайтовая адресация: по одному адресу можно получить только один байт данных.
2. Строковый доступ к основной памяти: минимальная единица чтения из основной памяти - строка.
3. Состояние строки кэша задаётся одним битом действительности.
4. Каждое чтение из основной памяти должно кэшироваться.
5. Результат чтения из памяти - отдельный регистр разрядностью 8 бит.

В модели должна быть возможность ввода адреса вручную и случайным образом. Строк в кэш-памяти должно быть 16, длина строки - 2 байта, адресуемая память - 512 байт. Также должна быть реализована тэговая директория, в которой будут храниться тэги строк, записанных в кэше. Тэговая директория и хранение битов действительности должны быть реализованы в виде отдельных регистров. Кэш-память также должна быть реализована в виде набора регистров.

С точки зрения функциональной последовательности схема должна функционировать следующим образом:

1. Оператор указывает адрес нужного ему слова (байта) в основной памяти (или адрес генерируется случайным образом).
2. Кэш-контроллер анализирует адрес и проверяет наличие соответствующей строки в кэш-памяти.
3. В случае попадания запрошенное слово (байт) загружается из кэш-памяти в регистр-получатель.
4. В случае промаха соответствующая строка читается из основной памяти, записывается в кэш-память, запрошенное слово (байт) загружается из кэш-памяти в регистр-получатель.

1.2 Ход выполнения работы

Посчитаем разрядность адреса, тэга, индекса и смещения. Размер адресуемой памяти = 512 байт, а значит разрядность адреса = 9. Длина строки = 2 байта, значит разрядность смещения = 1. Количество строк в кэш-памяти = $16 = 2^4$, значит разрядность индекса = 4. Оставшиеся биты адреса - это тэг, значит разрядность тэга = $9 - 1 - 4 = 4$.

Разместим на схеме контакт для ввода адреса и генератор случайных чисел, затем разместим мультиплексор, который будет выбирать адрес. Также добавим генератор тактового сигнала (Рис. 1).

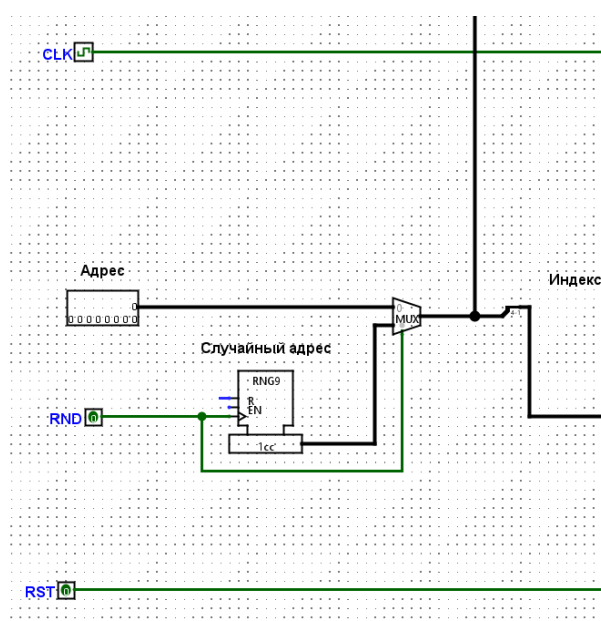


Рис. 1. Ввод адреса

Разместим на схеме основную память разрядностью 16 бит, т.к. длина строки 2 байта (Рис. 2). В обращении не используется смещение, потому что доступ осуществляется по строкам. Значит разрядность адреса ОЗУ = $9 - 4 = 5$ бит.

Создадим дополнительную схему, которая представляет собой хранение одной строки, тэга этой строки и бита валидности, а также логику сравнения тэгов и действительности данных (Рис. 3). На вход этой схемы подаются: полный адрес на контакт address; строка на контакт D; тактовый сигнал на контакт CLK; бит, сообщающий о том, что нужно записать строку в регистры кэша на контакт Write_enable; бит для сброса данных на контакт RST. Выходами из схемы являются контакты: Word - на него выводится запрашиваемое слово; hit- бит, который в состоянии «1» сообщает о том, что в кэше находятся нужные данные, в состоянии

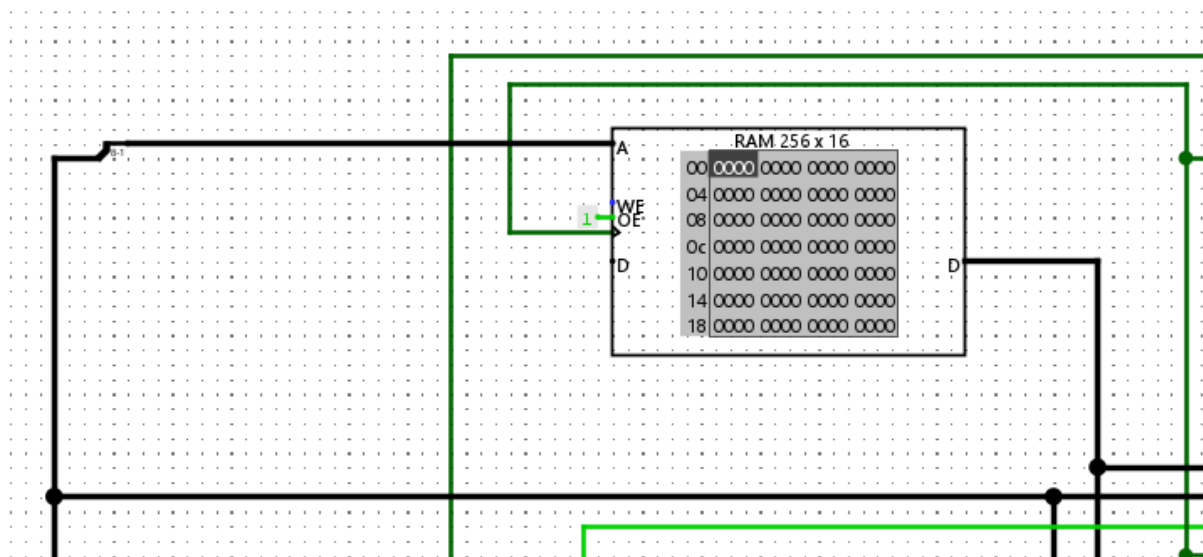


Рис. 2. Память

«0» сообщает обратное. Контакт Write_enable подключен к одноимённым контактам на всех регистрах схемы, аналогично к регистрам подключен контакт Reset. Адрес разделяется элементом «Разветвитель» на компоненты: тэг (4 бита) и смещение (1 бит). Далее тэг поступает на регистр тэговой директории, запись тэга происходит только при условии, что Write_enable и CLK находятся в состоянии «1». В компараторе сравниваются тэг, поступивший с регистра, и тэг, поступивший из адреса. Если тэги совпали, есть тактовый сигнал и бит действительности равен 1, то на выход hit подается 1. Бит действительности хранится в регистре и принимает значение «1» после первой записи данных в кэш. Если на контакты Write_enable и CLK подаются 1, то возможна запись строки в регистр. Регистр, хранящий строку, подключен к мультиплексору, из-за этого строка подается на выход Word, только если на выбирающий бит мультиплексора, являющийся смещением, подается 1.

На основную схему добавим 16 блоков схемы cache_memory, так как количество строк в кэш-памяти равно 16, а одна схема cache_memory хранит одну строку. Та схема, на которую подается строка, выбирается с помощью декодера. Также на схему добавим контакт Reset, с помощью которого можно сбрасывать состояние кэша. Основная логика работы схемы использует выходы hit из схем cache_memory, соединенные элементом «ИЛИ», результат после него также инвертируется, и тактовый сигнал. Если при включении тактового сигнала ни в одном блоке кэша нет попадания, то происходит обращение к основной памяти, из которой берётся запрашиваемая строка, которая записывается в блок кэша. Од-

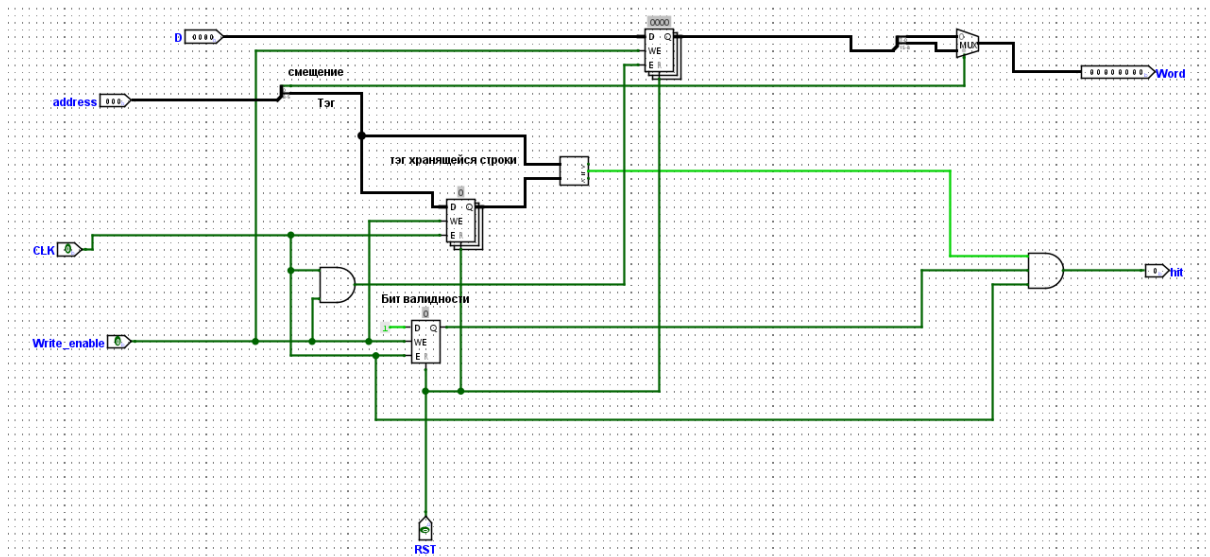


Рис. 3. Схема cache_memory

повременно с этим обновляется тэг, хранящийся в этом блоке кэша, и засчитывается попадание, запрашиваемое слово из кэша. Результат инвертирования и тактовый сигнал проходят через элемент «И», после чего результат направляется в память (Рис. 4).

Для того, чтобы вывести полученное слово, соединим выходы Word схем `cache_memory` с мультиплексором, его выбирающим битом будет индекс (Рис. 5). К регистру-получателю подключим выход мультиплексора, тактовый сигнал, сигнал сброса `Reset`, результат логического сложения выходов `hit` блоков кэша, чтобы гарантировать, что в регистр попадают только корректные данные.

Итоговая схема представлена в [Приложении А](#).

1.3 Заключение первой части работы

В первой части лабораторной работы была составлена схема, представляющая собой модель кэш-памяти прямого отображения со следующими характеристиками: 16 строк в кэш-памяти, длина строки - 2 байта, адресуемая память - 512 байт. В схеме обеспечена побайтовая адресация, строковый доступ к основной памяти. Состояние строки кэша задаётся одним битом действительности. Каждое чтение из основной памяти кэшируется. Результатом чтения из памяти является отдельный регистр разрядностью 8 бит.

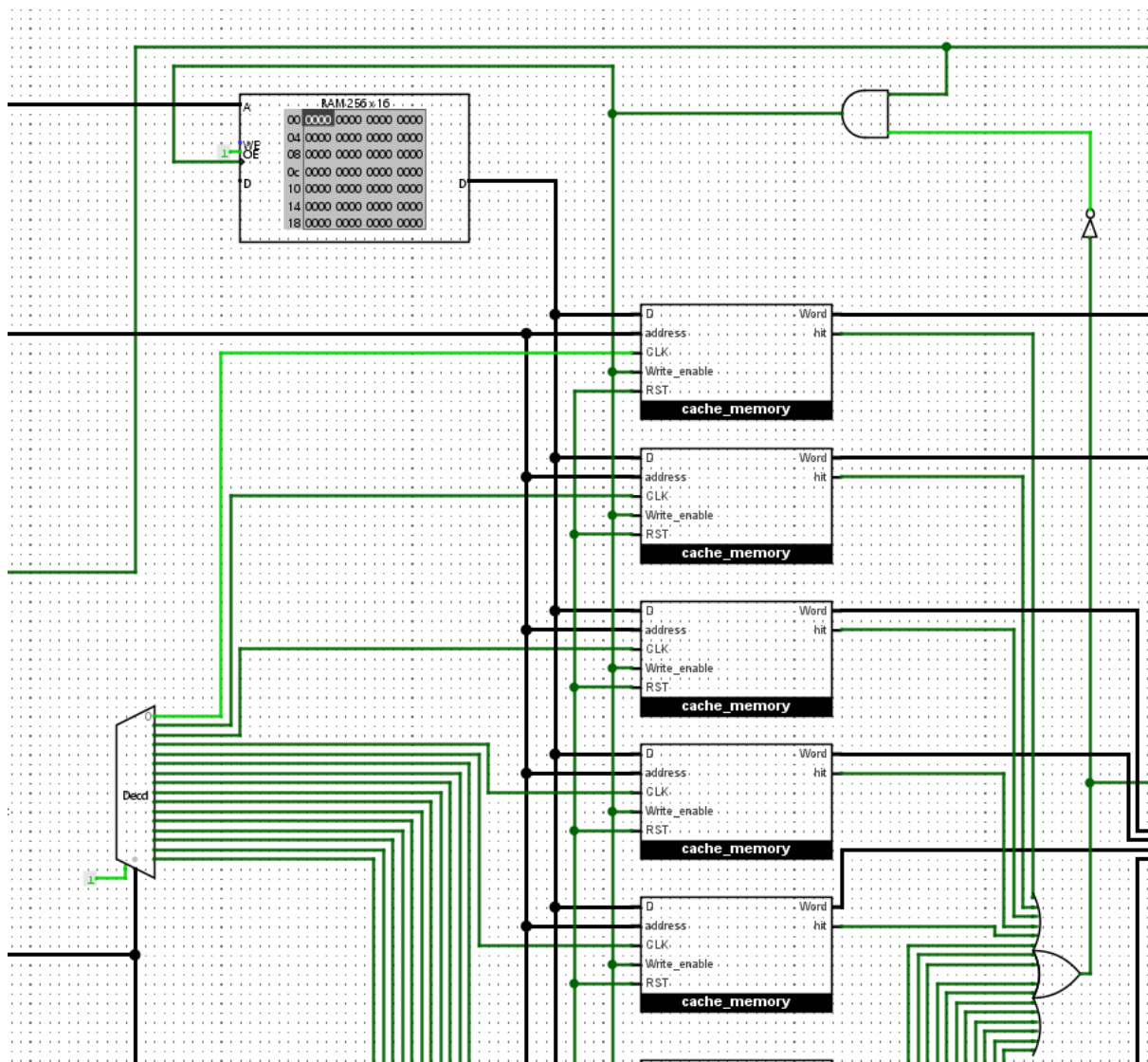


Рис. 4. Кэш и память

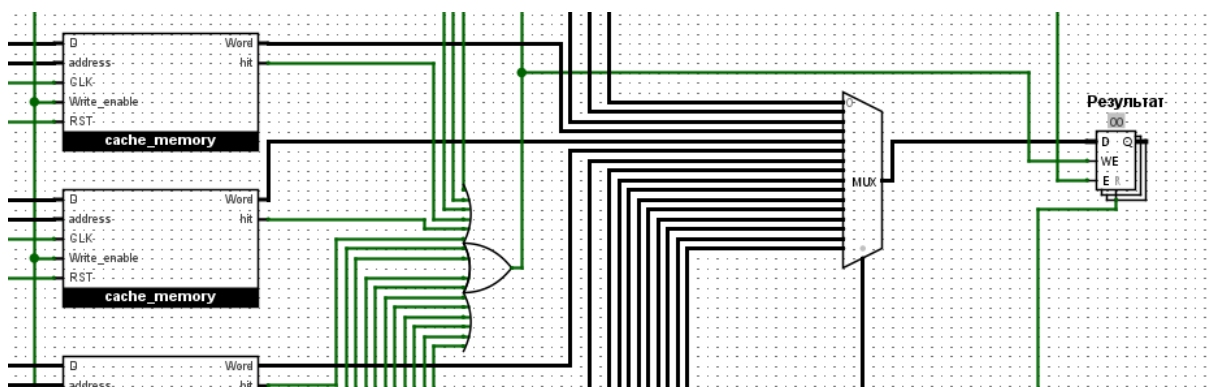


Рис. 5. Регистр-получатель

2 Вторая часть работы

2.1 Постановка задачи

Во второй части работы требуется изменить схему, составленную в первой части, так, чтобы было реализовано следующее:

1. Разделить линию тактирования, замедлив в 4 раза те ответвления, которые подключены к элементам ОЗУ;
2. При выполнении операции чтения слова по указанному адресу в целевой регистр сначала выполнять проверку наличия строки в кэше и в случае попадания читать слово из строки кэша, но при промахе продолжать обращение к основной памяти, читать строку оттуда, записывая её в кэш, а запрошенное слово- в целевой регистр.

2.2 Ход выполнения работы

Для того, чтобы замедлить линию тактирования, идущую к основной памяти, подключим счётчик с разрядностью 2 бита, таким образом счётчик может хранить 4 значения: 0, 1, 2 и 3. Добавим компаратор, к которому присоединим к одному из входов константу «0». К другому входу компаратора подключим выход счётчика. Выход компаратора, подключим к тому же элементу, к которому ранее был подключен тактовый сигнал напрямую (Рис. 6). Пункт 2 постановки задачи уже был выполнен на схеме из первой части работы.

Итоговая схема представлена в [Приложении Б](#).

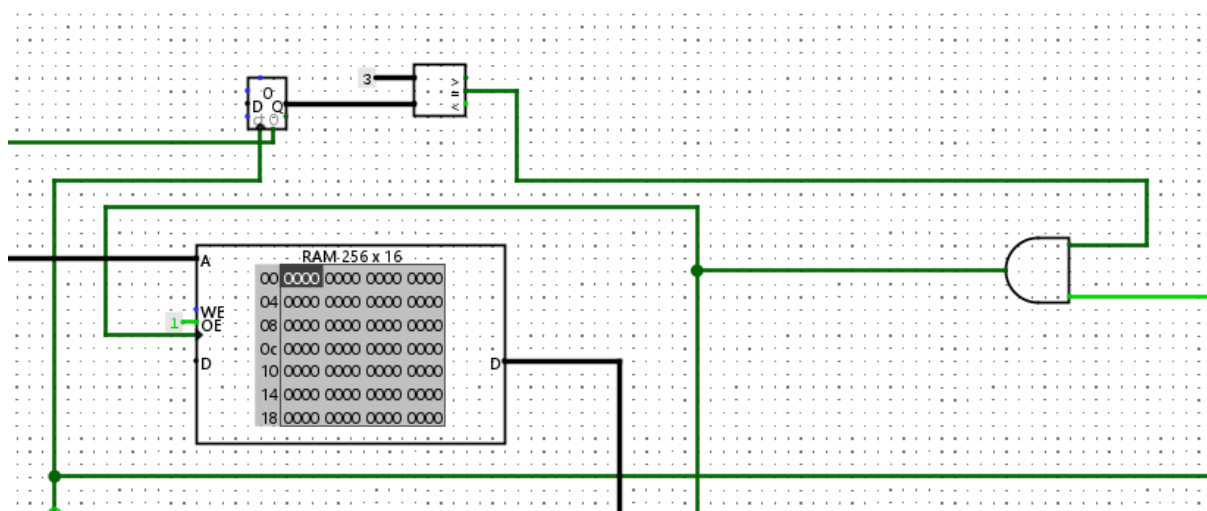


Рис. 6. Замедление линии тактирования

2.3 Заключение второй части работы

Во второй части лабораторной работы было реализовано замедление линии тактирования в 4 раза для элементов, подключенных к ОЗУ, а также такое свойство работы схемы, что при выполнении операции чтения слова по указанному адресу в целевой регистр сначала выполнять проверку наличия строки в кэше и в случае попадания читать слово из строки кэша, но при промахе продолжать обращение к основной памяти, читать строку оттуда, записывая её в кэш, а запрошенное слово - в целевой регистр.

Заключение

В ходе выполнения лабораторной работы была разработана модель кэш-памяти прямого отображения в среде Logisim-evolution. Работа состояла из двух частей, в каждой из которых решались поставленные задачи.

В первой части была реализована базовая схема кэш-памяти со следующими характеристиками:

1. 16 строк кэш-памяти длиной 2 байта каждая;
2. адресуемая память объёмом 512 байт;
3. побайтовая адресация при строковом доступе к основной памяти;
4. хранение состояния строки кэша с использованием бита валидности;
5. автоматическое кэширование каждого чтения из основной памяти;
6. вывод результата в 8-битный регистр-получатель.

Были рассчитаны разрядности адреса, тега, индекса и смещения, а также реализованы ручной и случайный ввод адреса, теговая директория и биты валидности в виде отдельных регистров, логика обработки попаданий и промахов в кэше.

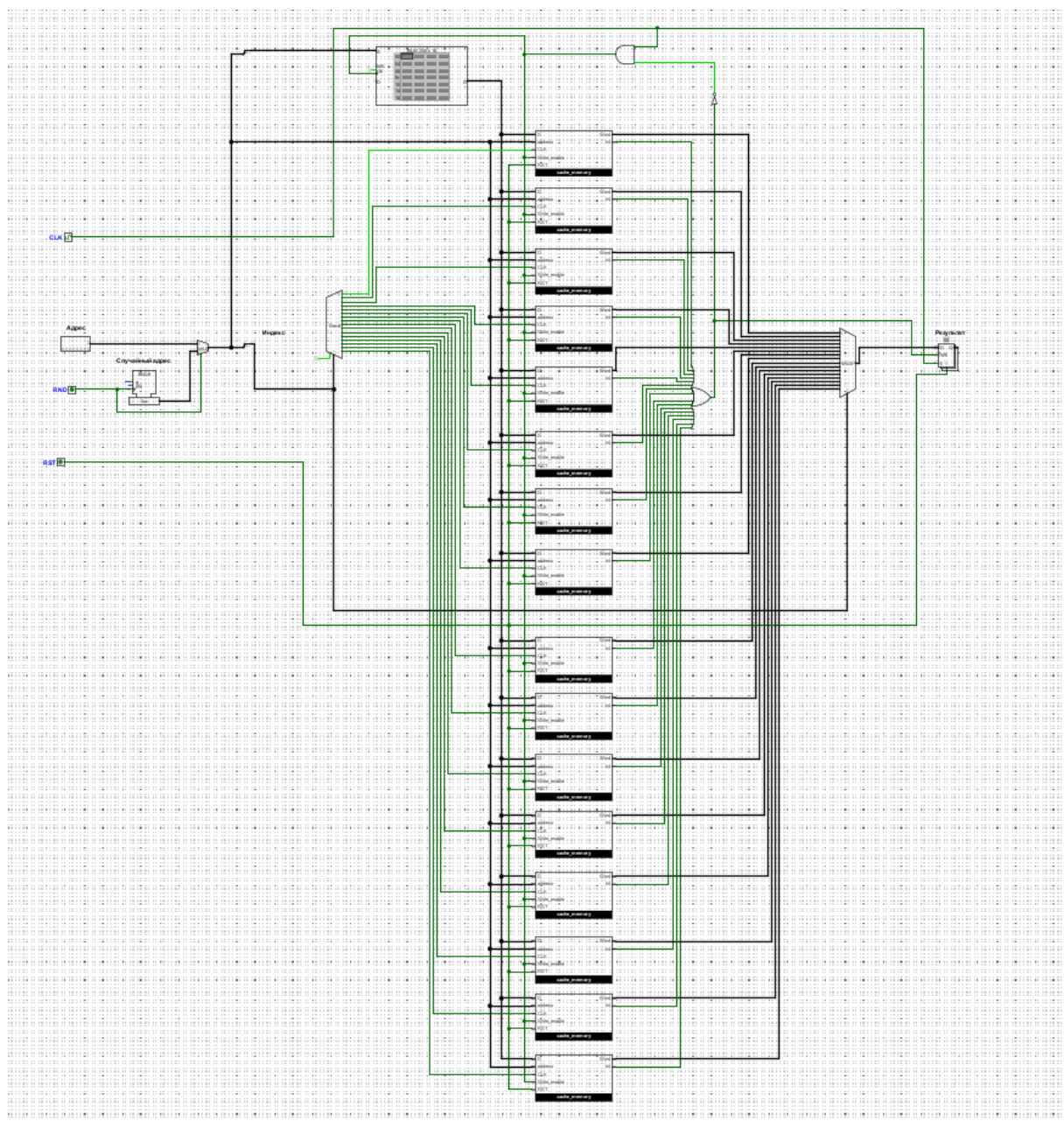
Во второй части в схему были внесены следующие изменения:

1. линия тактирования для ОЗУ замедлена в 4 раза с использованием счётчика и компаратора;
2. сохранён и оптимизирован алгоритм работы кэша, при котором при промахе данные сначала загружаются из основной памяти в кэш, а затем передаются в регистр-получатель.

Список использованных источников

1. ОЗУ / [Электронный ресурс] // cburch.com : [сайт]. — URL: <https://cburch.com/logisim/docs/2.5.0/ru/libs/mem/ram.html> (дата обращения: 22.05.2025).
2. Регистр / [Электронный ресурс] // cburch.com : [сайт]. — URL: <https://cburch.com/logisim/docs/2.5.0/ru/libs/mem/register.html> (дата обращения: 22.05.2025).
3. Декодер / [Электронный ресурс] // cburch.com : [сайт]. — URL: <https://cburch.com/logisim/docs/2.5.0/ru/libs/plexers/decoder.html> (дата обращения: 22.05.2025).
4. Мультиплексор / [Электронный ресурс] // cburch.com : [сайт]. — URL: <https://cburch.com/logisim/docs/2.5.0/ru/libs/plexers/mux.html> (дата обращения: 22.05.2025).

Приложение А



Приложение Б

